Figure 7.11 As opposed to the example in figure 7.10, an efficient implementation of an LFU cache, using a priority queue to keep track of which elements should be evicted next. Here the PriorityQueue (which is an abstract data type) is represented ad a binary heap. Note that we don't need to keep separate pointers for the points of insertion and eviction like for the linked list.

For sure, we can forget about constant-time insertion: that only works for the LRU. We would still maintain constant-time lookup; at least that, that's good.

But adding or updating a new element, both could take up to linear time, in some edge cases where most of the elements are used with the same frequency.

Can we do any better? Well, once again, at this point a bell should ring (if it doesn't, you might want to take a look at chapter 2 and appendix C before moving on).

We are no longer basing our eviction policy on a FIFO queue[24], the order of insertion doesn't matter anymore, instead we have a priority associated with each node.

Now, do you see where I am going? The best way to keep an order on entries based on their dynamically changing priority is, of course, a priority queue.

Listing 7.5 LFU Cache construction

```
function LFUCache(maxElements)
```